

Balanceador de Carga con Apache y Docker

Isabella Ortiz, Julián Viáfara, Samuel Sepúlveda,
Sebastián Cobos, Isabela Cabezas, Valentina Velastegui

Universidad Autónoma de Occidente

isabella.ortiz_her@uao.edu.co
julian.viafara@uao.edu.co
samuel.sepulveda@uao.edu.co
sebastian.cobos@uao.edu.co
isabela.cabezas@uao.edu.co
valentina.velastegui@uao.edu.co

I. INTRODUCCIÓN

Actualmente, las empresas buscan desarrollar aplicaciones en la nube que garanticen alta disponibilidad, escalabilidad y una rápida respuesta ante fallos, especialmente frente al aumento de usuarios y solicitudes concurrentes. En este contexto, el balanceo de carga se ha convertido en una técnica fundamental, ya que permite distribuir el tráfico entre múltiples servidores y evitar la saturación de un único nodo.

En este proyecto se implementó un clúster web utilizando balanceo de carga mediante el módulo *mod_proxy_balancer* de Apache HTTP Server. Para ello, se desplegaron tres servidores backend en contenedores independientes de Docker, orquestados mediante Docker Compose. Además, se realizaron pruebas de carga con Artillery para validar el comportamiento del sistema bajo diferentes niveles de demanda.

El objetivo principal de este trabajo es desarrollar un sistema capaz de soportar altos volúmenes de solicitudes simultáneas sin presentar fallos, evaluando su desempeño mediante métricas como la latencia, el *throughput* y la tasa de error.

II. CONTEXTO

Luego de la pandemia del COVID-19, muchos procesos comenzaron a digitalizarse de manera acelerada. Las empresas migraron gran parte de sus servicios a plataformas web, mientras que los usuarios empezaron a realizar actividades cotidianas de forma virtual, como compras, trabajo, estudios y trámites. Como consecuencia, las aplicaciones y servicios web tuvieron que adaptarse para soportar un volumen de solicitudes mucho mayor al esperado.

Este incremento del tráfico generó nuevos desafíos para las infraestructuras tecnológicas, ya que los usuarios demandan tiempos de respuesta rápidos, disponibilidad continua y estabilidad del sistema. Sin embargo, las arquitecturas tradicionales basadas en un único servidor presentan limitaciones importantes al enfrentar grandes cantidades de peticiones simultáneas.

Entre los principales problemas se encuentran la sobrecarga del servidor, el aumento de la latencia, la saturación de recursos y la existencia de puntos únicos de falla. Esto provoca baja tolerancia a fallos, dificultades de escalabilidad e interrupciones del servicio, afectando negativamente la experiencia de los usuarios.

III. ALTERNATIVAS DE SOLUCIÓN

Teniendo en cuenta la problemática planteada se consideraron algunas alternativas de solución utilizando la metodología de balanceo de carga como:

- Balanceador de carga web con NGINX que ofrece un manejo eficiente de conexiones con un bajo consumo de memoria, además proporciona una gran versatilidad ya que puede funcionar como servidor web, cache, balanceador y proxy inverso.
- Balanceo de carga de carga web con HAProxy que es una solución especializada en balanceo de carga y alta disponibilidad. Destaca por su precisión en la distribución de tráfico, monitoreo de nodos backend y soporte para sesiones persistentes, siendo una herramienta común en entornos empresariales.
- Balanceador de carga web con Apache *mod_proxy_balancer* esta fue la solución seleccionada para este informe, ya que nos permite crear una solución versátil, flexible, integrada y persistente. Debido a que soporta múltiples protocolos, también soporta sesiones basadas en cookies o codificación URL.

Criterio de evaluación	NGINX	HAProxy	Apache HTTP Server
Enfoque principal	Optimización del rendimiento web y gestión de conexiones concurrentes	Alta disponibilidad y distribución inteligente del tráfico	Integración de servicios web y balanceo mediante módulos
Característica destacada	Bajo consumo de memoria y múltiples funciones integradas	Monitoreo de nodos y precisión en la distribución de carga	Soporte multiprotocolo y persistencia de sesiones
Administración de sesiones	Configurable mediante cookies y reglas de afinidad	Soporte nativo para sesiones persistentes	Soporte mediante cookies y codificación URL
Integración con infraestructura	Ideal para arquitecturas modernas y microservicios	Orientado a entornos empresariales de alto tráfico	Integración directa con ecosistemas basados en Apache
Facilidad de despliegue en Docker	Configuración ligera y rápida	Requiere configuración específica del tráfico	Integración sencilla con contenedores y servicios web

IV. DISEÑO DE LA SOLUCIÓN

Se planteó e implementó una arquitectura distribuida que emplea contenedores Docker para resolver la problemática presentada. Para equilibrar la carga, se utilizó como componente principal Apache HTTP Server con `mod_proxy_balancer`. La solución se organizó a partir de un método de disponibilidad alta y división de responsabilidades, lo que posibilitó la asignación eficaz de las solicitudes entre los diversos servicios y nodos del sistema. La arquitectura aplicada al sistema consistió en dividirlo en dos niveles principales de balanceo: balanceador del backend y balanceador del frontend.

Como primer nivel, correspondiente al frontend, se desplegó un balanceador de carga utilizando Apache HTTP Server, con el propósito de actuar como punto principal de entrada para las solicitudes de los usuarios. De esta manera, el sistema puede recibir y distribuir eficientemente todas las peticiones HTTP dirigidas a esta capa. Bajo esta arquitectura, se implementaron dos instancias independientes (Aplicación Frontend 1 y Aplicación Frontend 2), cada una con la aplicación web integrada de CybersecurityLab, desplegadas en contenedores Docker separados. Esta estrategia permite evitar que un único frontend se convierta en un punto de saturación o de fallo, mejorando así la disponibilidad y la tolerancia a fallos del sistema.

El balanceador backend, que también se aplica con `mod_proxy_balancer` de Apache HTTP Server, es el segundo nivel de la arquitectura. Este componente opera como una API Gateway, que se encarga de recibir las solicitudes que vienen de los frontends y encaminarlas a los distintos microservicios del backend, dependiendo del endpoint requerido. Así, el sistema tiene la capacidad de distribuir las solicitudes entre varios servicios especializados de manera dinámica.

Además, el backend del sistema también fue dividido en servicios independientes siguiendo una arquitectura basada en microservicios, cada microservicio fue implementado en contenedores Docker distintos para poder administrarlos de manera independiente. Con la división de estos servicios se

buscó escalar módulos específicos según la demanda, facilitar el mantenimiento de la aplicación y aumentar la tolerancia a fallos.

Es importante destacar que estos servicios backend comparten una base de datos centralizada para guardar datos sobre los usuarios y las publicaciones.

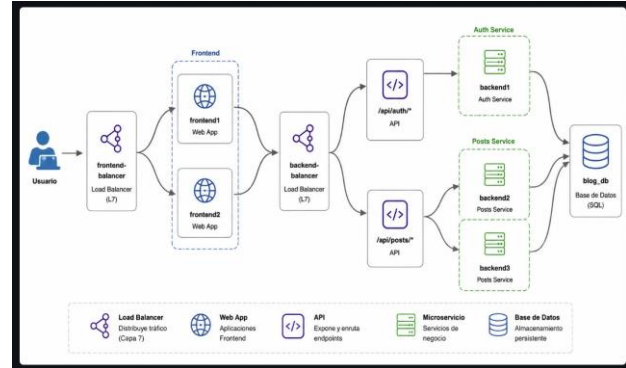


Figura 1. Diseño de la arquitectura

Para distribuir el tráfico de peticiones en los diferentes nodos descritos anteriormente se utilizó como algoritmos de balanceo: Round Robin (byrequests), el cual distribuye las peticiones de conexión a los servidores web en la secuencia en que llegan, este se utiliza principalmente cuando los servidores tienen capacidades de procesamiento y almacenamiento casi iguales. Y Least Connections (bybusyness) el cual le prioridad a los servidores que tengan menos conexiones activas y, de acuerdo con esto, asigna las peticiones de los usuarios entrantes.

Por último, para el desarrollo de la solución es importante destacar que toda la implementación e infraestructura fue desarrollada en Docker y Docker compose posibilitando la automatización de la creación, ejecución y comunicación de varios servicios. Cada parte de la arquitectura se implementó en su contenedor específico. Además, se emplearon redes Docker internas y variables de entorno que pueden ser configuradas para simplificar la implementación y la expansión horizontal del sistema.

V. IMPLEMENTACIÓN

Como se mencionó anteriormente, la implementación se llevó a cabo empleando una arquitectura distribuida basada en contenedores Docker, posibilitando la gestión y el despliegue de cada parte del sistema de manera autónoma. Para el desarrollo del sistema se utilizó Vagrant para la automatización

de las máquinas virtuales, VirtualBox para la virtualización, Docker para la contenerización de servicios, Docker Compose para la orquestación de contenedores, Apache HTTP Server con mod_proxy_balancer para el balanceador de carga, Node.js + Express como servicios backend, React + Vite para la aplicación frontend y MySQL para la persistencia de datos.

1) Se creó un entorno virtualizado a través de un Vagrantfile, que define una máquina virtual 'servidor' con Ubuntu 22.04. En esta máquina se despliega toda la infraestructura Dockerizada, incluyendo los balanceadores, backends, frontends y la base de datos. Las pruebas de carga se ejecutan desde un contenedor de Artillery desplegado dentro de la misma red Docker del proyecto.



```

Vagrantfile

Vagrant.configure("2") do |config|
  config.boot_timeout = 600

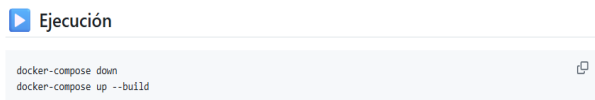
  # SERVIDOR
  config.vm.define "servidor" do |servidor|
    servidor.vm.box = "ubuntu/jammy64"
    servidor.vm.hostname = "servidor"
    servidor.vm.network "private_network", ip: "192.168.50.3"
    servidor.vm.provider "virtualbox" do |vb|
      vb.memory = 2048
      vb.cpus = 2
    end
  end

  # CLIENTE
  config.vm.define "cliente" do |cliente|
    cliente.vm.box = "ubuntu/jammy64"
    cliente.vm.hostname = "cliente"
    cliente.vm.network "private_network", ip: "192.168.50.2"
    cliente.vm.provider "virtualbox" do |vb|
      vb.memory = 1024
      vb.cpus = 1
    end
  end
end

```

Figura 2. Vagrantfile

2) Se utilizó el archivo docker-compose.yml para la orquestación de todos los servicios. Este archivo es responsable de: generar de manera automática todos los servicios, configurar redes internas de Docker, administrar variables de entorno, establecer volúmenes que persistan y promover el escalado horizontal.



```

Ejecución

docker-compose down
docker-compose up --build

```

Figura 3. Levantamiento y creación de contenedores

3) Las instancias frontend se crearon con Vite y React, y se sirvieron en contenedores Docker independientes. Se utilizó mod_proxy_balancer de Apache para establecer la configuración del balanceador frontend, distribuyendo las solicitudes entre las instancias disponibles mediante el algoritmo Round Robin.

4) Se utilizó Apache HTTP Server como puerta de enlace API para implementar el balanceador de backend. Este componente recibe las peticiones que llegan desde el frontend y las encamina de manera dinámica hacia los microservicios adecuados, de acuerdo con el endpoint requerido.

Las rutas configuradas fueron:

Ruta	Destino
/api/auth/*	backend1
/api/blog/*	backend2 backend3

Al principio, la aplicación operaba como un modelo monolítico en el que todas las funciones estaban incorporadas en un solo backend. No obstante, este método tenía limitaciones significativas en cuanto al rendimiento y la capacidad de escalar. Para resolver este inconveniente, se dividió el backend en microservicios autónomos: backend1: Autenticación, Backend2: Blog Service, Backend3: Blog Service replicado. El servicio de autenticación tuvo la responsabilidad de: Inscripción de los usuarios, sesión de inicio, producción de tokens JWT y cifrado de contraseñas con bcrypt. Por otra parte, para gestionar el contenido y las publicaciones del blog se emplearon los servicios backend2 y backend3.

VI. PRUEBAS

Se llevaron a cabo diversas pruebas de rendimiento y funcionalidad en los balanceadores frontend y backend con el fin de verificar que la arquitectura implementada funcione correctamente. Las pruebas facilitaron la comprobación de cómo se comportaba el sistema a diferentes niveles de concurrencia, así como su tolerancia a fallos y la distribución del tráfico. Artillery, ejecutado en un contenedor Docker específico dentro de la red del proyecto, fue utilizado para las pruebas de carga. Se guardaron los resultados obtenidos en archivos .json para analizarlos más adelante.

1) La primera validación fue verificar si el balanceador frontend repartía las peticiones de manera adecuada entre las dos instancias de la aplicación web.

Frontend:



```

for i in {1..10}; do curl http://192.168.50.3:8090; echo ""; done

```

Figura 4. Prueba de validación al frontend

2) Posteriormente, se validó el funcionamiento del balanceador backend y el API Gateway encargado de enrutar solicitudes hacia los microservicios correspondientes.

Backend:

```
for i in {1..10}; do curl http://192.168.50.3:8080; echo ""; done
```

Figura 5. Prueba de validación al backend

3) También se realizaron pruebas para validar el estado de disponibilidad de los servicios backend mediante endpoints de health check.

Health check:

```
for i in {1..10}; do curl http://192.168.50.3:8080/health; echo ""; done
```

Figura 6. Prueba de endpoints de health check

4) Con el propósito de examinar cómo funciona el sistema en diferentes situaciones de concurrencia, se crearon varios scripts de pruebas con Artillery.

Archivo	Escenario
artillery/balanceador-50.yml	50 usuarios simultáneos
artillery/balanceador-100.yml	100 usuarios simultáneos
artillery/balanceador-500.yml	500 usuarios simultáneos
artillery/balanceador-1000.yml	1000 usuarios simultáneos
artillery/balanceador-5000.yml	5000 usuarios simultáneos
artillery/balanceador-failover.yml	Prueba de caída del backend

Figura 7. Scripts de pruebas con Artillery para diferentes escenarios

Se examinaron las métricas siguientes a lo largo de cada prueba: Throughput: número de solicitudes que se procesan por segundo. Tiempo promedio de respuesta. p95 El tiempo de respuesta para el 95% de las solicitudes. p99 Tiempo de respuesta del 99 % de las peticiones. HTTP 200: Número de respuestas exitosas. Solicitudes fallidas y cantidad de solicitudes que no han tenido éxito

VII. DISCUSIÓN DE LAS PRUEBAS

Los resultados de las pruebas de carga se resumen en las siguientes tablas:

Prueba	Requests	HTTP 200	Completados	Fallidos	Req/s	Mean	p95	p99
50 usuarios	936	919	33	17	27/s	9.6 ms	32.8 ms	49.9 ms
100 usuarios	2.831	2.831	72	0	56/s	6.1 ms	18 ms	80.6 ms
500 usuarios	2.708	2.591	10	108	57/s	1044.7 ms	25.8 ms	50.9 ms
1000 usuarios	19.714	19.564	586	145	228/s	4066.3 ms	450.4 ms	2231 ms
5000 usuarios	25.000	25.000	5000	0	187/s	4.9 ms	18 ms	47.9 ms
Failover	1200	1200	1200	0	19/s	1.4 ms	3 ms	6 ms

Figura (). Resumen de resultados de las pruebas con algoritmo Round Robin

Prueba	Requests	VUsers completados	Fallidos	HTTP 200	Req/s	Mean	p95	p99
50 usuarios	1.500	50	0	1.500	32/s	3.0 ms	7.9 ms	15 ms
100 usuarios	3.000	100	0	3.000	61/s	2.4 ms	6 ms	12.1 ms
500 usuarios	15.000	500	0	15.000	260/s	22.6 ms	98.5 ms	214.9 ms
1000 usuarios	29.932	988	12	29.820	387/s	63.7 ms	202.4 ms	278.7 ms
5000 usuarios	25.000	5000	0	25.000	199/s	2.8 ms	10.1 ms	32.8 ms
Failover	1.200	1200	0	1.200	19/s	1.4 ms	3 ms	6 ms

Figura (). Resumen de resultados de las pruebas con algoritmo Least Connections

1) Escenario de 50 usuarios simultáneos: en este escenario inicial ambos algoritmos mostraron un comportamiento estable y sin una carga significativa sobre la infraestructura.

Con Round Robin, se procesaron 936 solicitudes, con 919 respuestas HTTP 200, una latencia promedio de 9.6 ms y un throughput de 27 solicitudes por segundo. Sin embargo, se registraron 17 solicitudes fallidas.

Por otro lado, Least Connections procesó 1500 solicitudes, completando los 50 usuarios virtuales sin errores, alcanzando 32 solicitudes por segundo y una latencia promedio de solo 3.0 ms.

2) Escenario de 100 usuarios simultáneos:

Round Robin procesó 2831 solicitudes con una latencia promedio de 6.1 ms y un throughput de 56 solicitudes por segundo, sin registrar errores.

En comparación, Least Connections procesó 3000 solicitudes, completó la totalidad de usuarios sin fallos y alcanzó 61 solicitudes por segundo con una latencia promedio de 2.4 ms.

En este escenario, Least Connections mantuvo mejores tiempos de respuesta y un rendimiento ligeramente superior.

3) Escenario de 500 usuarios simultáneos: en cargas intermedias comenzaron a evidenciarse diferencias más marcadas entre ambos algoritmos.

Round Robin procesó 2708 solicitudes, de las cuales 108 fallaron. Además, la latencia promedio superó los 1000 ms, con un throughput de 57 solicitudes por segundo.

Por su parte, Least Connections procesó 15000 solicitudes, completando los 500 usuarios sin errores, alcanzando 260 solicitudes por segundo y una latencia promedio de 22.6 ms.

Los resultados muestran que Least Connections distribuyó la carga de manera más eficiente, evitando saturación en nodos específicos.

4) *Escenario de 1000 usuarios simultáneos*: este escenario representó una de las pruebas más exigentes para la infraestructura.

Con Round Robin, se procesaron 19714 solicitudes, registrando 145 fallos y una latencia promedio de 4066.3 ms, aunque se alcanzó un throughput de 228 solicitudes por segundo. Esto evidenció degradación en el rendimiento bajo alta concurrencia.

En contraste, Least Connections procesó 29932 solicitudes, con solo 12 fallos, alcanzando 387 solicitudes por segundo y una latencia promedio de 63.7 ms.

En este escenario, Least Connections mostró una ventaja significativa en escalabilidad y distribución dinámica de carga.

5) *Escenario de 5000 usuarios simultáneos*: En la prueba de máxima concurrencia, ambos algoritmos mantuvieron disponibilidad total del servicio.

Round Robin completó 25000 solicitudes sin errores, con una latencia promedio de 4.9 ms y un throughput de 187 solicitudes por segundo.

De forma similar, Least Connections también completó 25000 solicitudes sin fallos, alcanzando 199 solicitudes por segundo y una latencia promedio de 2.8 ms.

Aunque ambos algoritmos mostraron resultados sobresalientes, Least Connections presentó una ligera mejora en rendimiento y latencia.

6) *Prueba de tolerancia a fallos*: Por último, se llevó a cabo una prueba de tolerancia a fallos que simulaba la caída de uno de los servicios del backend. El propósito fue comprobar que el balanceador tuviera la capacidad de encaminar el tráfico hacia los nodos disponibles sin perjudicar en gran medida el servicio.

Tanto con Round Robin como con Least Connections, las 1200 solicitudes fueron completadas exitosamente, sin errores y con una latencia promedio de 1.4 ms.

Esto confirmó que la configuración de failover implementada con Apache HTTP Server y `mod_proxy_balancer` funciona correctamente independientemente del algoritmo de balanceo utilizado.

VIII. CONCLUSIONES

- La implementación de una arquitectura de balanceo de carga permitió distribuir las solicitudes entre múltiples nodos, eliminando la dependencia de un único servidor y reduciendo los puntos únicos de fallo dentro del sistema.
- La división del balanceo en dos niveles, frontend y backend, mejoró la disponibilidad, la tolerancia a fallos y la capacidad de escalamiento horizontal de la infraestructura implementada.
- El uso de Docker, Docker Compose y contenedores independientes facilitó el despliegue, la orquestación y la administración de todos los componentes del sistema de manera automatizada.
- La separación del backend bajo una arquitectura de microservicios permitió una administración más flexible de los servicios de autenticación y publicaciones, facilitando el mantenimiento y la escalabilidad de módulos específicos.
- Las pruebas realizadas con Artillery demostraron que la arquitectura implementada mantiene estabilidad, baja latencia y un throughput adecuado en escenarios de baja y media concurrencia.
- Ambos algoritmos de balanceo, Round Robin (byrequests) y Least Connections (bybusyness), mostraron un funcionamiento correcto en escenarios de 50 y 100 usuarios simultáneos, manteniendo tiempos de respuesta bajos y una distribución adecuada de las solicitudes.
- En escenarios de alta concurrencia, entre 500 y 1000 usuarios simultáneos, se observaron diferencias significativas entre ambos algoritmos. Mientras Round Robin presentó incrementos considerables en la latencia y un mayor número de solicitudes fallidas, Least Connections mantuvo un mejor rendimiento, menor latencia y una distribución de carga más eficiente.
- En la prueba de 5000 usuarios simultáneos, ambos algoritmos completaron exitosamente todas las solicitudes sin errores, confirmando que la arquitectura diseñada tiene la capacidad de soportar cargas elevadas bajo condiciones de alta demanda.
- Las pruebas de tolerancia a fallos evidenciaron que la configuración implementada con Apache HTTP Server y `mod_proxy_balancer` fue capaz de redirigir correctamente el tráfico ante la caída de un nodo backend, garantizando la continuidad del servicio.

- De manera general, los resultados obtenidos permitieron concluir que Least Connections ofrece un mejor comportamiento en escenarios con alta concurrencia, mientras que Round Robin representa una alternativa estable y funcional para cargas moderadas y distribuciones homogéneas.

IX. REFERENCIAS

[1] VMware, “Round Robin Load Balancing,” VMware. [En línea]. Disponible en: <https://www.vmware.com/topics/round-robin-load-balancing>. [Accedido: 15-may-2026].

[2] F5 Networks, “Least Connection,” F5. [En línea]. Disponible en: <https://www.f5.com/glossary/least-connection>. [Accedido: 15-may-2026].

[3] IBM, “What Is Load Balancing?,” IBM Think. [En línea]. Disponible en: [IBM Think – Load Balancing](#). [Accedido: 15-may-2026].

[4] GeeksforGeeks, “Issues Related to Load Balancing in Distributed System,” GeeksforGeeks. [En línea]. Disponible en: <https://www.geeksforgeeks.org/operating-systems/issues-related-to-load-balancing-in-distributed-system/>. [Accedido: 15-may-2026].

[5] Apache Software Foundation, “Apache Module mod_proxy_balancer,” Apache HTTP Server Documentation. [En línea]. Disponible en: [Apache HTTP Server Documentation](#). [Accedido: 15-may-2026].

[6] Docker Inc., “Docker Documentation,” Docker Docs. [En línea]. Disponible en: [Docker Documentation](#). [Accedido: 15-may-2026].